

Projektuppgift

Programmering i Typescript, DT208G

Projektuppgift

Programmering i Typescript, DT208G

Moa Brunemo



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Moa Brunemo, mobr2500@student.miun.se

Utbildningsprogram: Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: 02, 2026

Sammanfattning

I denna projektuppgift har en webbplats för kursöversikt och ramschema utvecklats med hjälp av TypeScript och Angular ramverk. Användaren kan lägga till och ta bort kurser, se total högskolepoäng samt antal kurser som för närvarande visas.

På webbplatsen kan användarna göra detta, samt även se hur många kurser de har i ramschemat när de befinner sig på kursöversikten. Även en pagination har lagts till för att hantera hur många kurser som visas – det finns i kursdatan över 4000 kurser, vilket kan bli mycket för en webbplats att läsa in på en gång.

I slutändan analyseras resultatet av detta projekt: vad som gick bra, vad som gick dåligt samt vad som kan tas med in i framtiden.

Innehållsförteckning

Sammanfattning	2
Introduktion / Inledning	4
1.1 Bakgrund och problemmotivering	4
1.2 Övergripande syfte	4
1.3 Avgränsningar	4
1.4 Mål med uppgiften	5
2 Teori / Bakgrundsmaterial	6
3 Metod	7
3.1 Design	7
3.2 Verktyg och arbetsprocess	9
3.3 Testning	9
4 Konstruktion	10
4.1 Hantering av data (JSON-fil och service)	10
4.2 Navigering och routing	10
4.3 Kursöversikt och Ramschema	11
4.3.1 Kursöversikt	11
4.3.2 Ramschema	12
5 Resultat	13
6 Analys	15
6.1 Struktur av webbplatsen	15
6.2 Funktionalitet	16
6.3 In i framtiden	17
7 Källförteckning	18

Introduktion / Inledning

Projektuppgiften i kursen TypeScript har gått ut på att skapa en webbapplikation som visar kurser som gick att studera under 2023 i tabellformat. Från denna tabell ska det vara möjligt att lägga till enskilda kurser till ett eget ramschema som ska gå att se på en annan sida i applikationen.

Applikationen är utvecklad med TypeScript och Angular för att möjliggöra en dynamisk användarupplevelse och skapa en lättunderhållen projektstruktur.

1.1 Bakgrund och problemmotivering

Som student kan det ibland verka som en omöjlig uppgift att kunna se över vilka kurser som ges ut inom vissa ämnen. Denna projektuppgift är både en utmaning och ett mål för att underlätta detta: att skapa en lösning som samlar kursinformation och gör det möjligt att visualisera ett personligt ramschema.

Minst två olika sidor kommer att behövas: en för att läsa ut samtliga kurser från 2023, en för att visa de utvalda kurserna. Vidare behövs minst två service-komponenter för att kunna hantera dessa två.

För kursöversikten behövs det funktionalitet för att kunna söka, filtrera efter ämne och sortera innehållet i fallande eller stigande ordning. Dessutom möjlighet att spara kurser till ramschemat (med exempelvis localStorage).

Ramschemat behöver också funktionalitet för att kunna visa fallande/stigande ordning samt att kunna ta bort kurser. Utöver detta ska det gå att utlösa totala summan för högskolepoäng av samtliga kurser gå att utläsa.

1.2 Övergripande syfte

Syftet med webbapplikationen är att ge användaren en tydlig och överskådlig struktur över kurser, där information enkelt kan organiseras, visas och uppdateras. Tanken med applikationen är att underlätta planering av studier genom att låta användaren själv bygga sitt ramschema utifrån det befintliga kursutbudet från 2023.

1.3 Avgränsningar

Kursutbudet är avgränsat till de kurser som gick att ta del av under år 2023 och fokuserar enbart på visning och hantering av denna information i applikationen. Ingen webbserver eller databas har använts, utan kursinformationen har hanterats lokalt i applikationen med hjälp av localStorage.

Vidare finns inte funktionalitet för inloggning eller användarkonto på något vis. Applikationen är inte kopplad till faktiska antagningssystem utan fungerar mer som en lista som man kan bygga en egen kurslista ifrån.

1.4 Mål med uppgiften

Målet med projektet att skapa en applikation som kan hantera information på ett effektivt sätt och som är användarvänlig. Detta betyder:

- Visa kurser i en tabell
- Gör det möjligt att enskilda kurser läggs till i ett personligt ram-schema
- Låt användaren navigera mellan kursöversikt och ramschema
- Säkerställ att data uppdateras korrekt vid interaktion
- Läs totalt antal högskolepoäng av de valda kurserna
- Visa totala antalet kurser utifrån filterning/sökning

Applikationen ska vidare vara responsiv och visas korrekt på stora och små skärmar, samt vara byggd med Angulars komponentbaserade struktur.

2 Teori / Bakgrundsmaterial

TypeScript är ett programmeringsspråk som bygger vidare på JavaScript och används för att skapa mer strukturerad och typsäker kod. Till skillnad från JavaScript så har TypeScript stöd för statiska typer vilket gör det möjligt att upptäcka fel i koden redan under utvecklingsfasen. Detta ökar kodens kvalitet och underlättar underhållet av applikationen. [3]

Angular är ett ramverk för webbutveckling som används för att skapa dynamiska webbapplikationer. Det möjliggör navigering mellan olika sidor utan att hela sidan behöver laddas om. Angular bygger på en komponentbaserad arkitektur, vilket innebär att applikationen delas upp i mindre, återanvändbara delar. [4]

Angulars Routing används för att navigera mellan olika vyer. Navigeringen kan ske med hjälp av **routerLink**, vilket gör det möjligt att länka till olika komponenter utan att ladda om sidan.

Signaler är ett reaktivt sätt att hantera tillstånd i Angular. De gör att endast de delar av gränssnittet som påverkas av en förändring uppdateras, istället för hela komponenten. Utan signaler hade Angular behövt identifiera på egen hand vad som har förändrats, vilket kan påverka prestandan på webbplatsen [1].

Computed signals används för att beräkna värden baserat på andra signaler. De uppdateras automatiskt när deras beroenden förändras, vilket gör att det beräknade värdet alltid är aktuellt utan manuell uppdatering. Computed används för att automatiskt räkna ut exempelvis totala värdet av något.

Control flow-syntax i Angular används för att styra hur innehåll visas i gränssnittet baserat på villkor eller för att upprepa element. Detta gör det möjligt att exempelvis visa eller dölja innehåll beroende på ett specifikt tillstånd, samt att rendera listor av data direkt i HTML.

Exempelvis **@if** används för villkorsstyrd rendering och **@for** för att iterera över listor och generera upprepade element i gränssnittet [2].

I projektet har samtliga ämnen ovan använts för att projektet ska få de funktioner och beteende som eftersökts. Signaler och control flow har varit viktiga för att visa för användaren när en förändring har skett, medan routing har gjort det mer behagligt och smidigt att byta vy mellan komponenterna.

3 Metod

Webbplatsen i detta projekt planerades att bestå av två huvudsidor: en kursöversikt och ett ramschema. För att möjliggöra navigering mellan dessa planerades det även en separat navigationskomponent. Innan utvecklingen av applikationen påbörjades gjordes en övergripande planering av applikationens struktur och funktionalitet.

För att inleda projektet skapades ett nytt Angular-projekt genom att i Git Bash skriva använda Angular CLI Detta gav en färdig utvecklingsmiljö med den grundstruktur och de filer som behövdes för den fortsatta utvecklingen av webbapplikationen.

3.1 Design

Ett par wireframes har skapats för att planera webbplatsens grundläggande design. Inget fokus lades på en mer avancerad visuell design i detta skede, utan på att säkerställa att användaren enkelt kan läsa och interagera med de olika kursobjekten. De mest relevanta värdena valdes ut och arrangerades hur de skulle presenteras som.

En tabell planerades att användas redan från början för att kunna läsa ut kursöversikten och ramschemat på ett överskådligt vis:

Kursöversikt Mitt ramschema

Kursöversikt 2023

Sök Väll ämne

Kurskod	Kursnamn	Poäng	Nivå	Ämne	Lägg till

Figur 1: wireframe av startsidan/kursöversikt

Kursöversikt <u>Mitt ramschema</u>					
Mitt ramschema					
Totala högskolepoäng: 20					
Kurskod ↕	Kursnamn ↕	Poäng ↕	Nivå ↕	Ämne ↕	Ta bort

Figur 2: wireframe av ramschemat

Det enda problemet med denna design kan bli att tabellen spränger ut på sidorna när skärmen blir mindre. Det bör kunna åtgärdas genom att inkorporera scroll funktion med css under utvecklingen dock:

Kursöversikt <u>Mitt ramschema</u>		
Sök		
Välj ämne		
Kurskod	Kursnamn	Poäng

Figur 3: mobilversion av kursöversikt, ramschema ska ha ett liknande

3.2 Verktyg och arbetsprocess

Som tidigare nämnt har både Angular och TypeScript använts i detta projekt. Utöver detta har Visual Studio Code, Git Bash samt GitHub använts för att utveckla, hantera och versionshantera projektarbetet.

Projektet byggdes stegvis: nästa del i arbetet inleddes enbart när föregående fungerade som planerat för att undvika möjliga problem. Komponenter och service var först på plats och fick grundläggande funktionalitet, innan service för att läsa in kursdata implementerades.

Mot slutet lades mer fokus på funktionalitet och interaktion för användaren som att filtrera, söka och lägga till kurser för de två huvudkomponenterna. Att kunna visa hur många kurser som visas, totala högskolepoäng samt antalet kurser placerade i ramschemat kom också till på slutet.

3.3 Testning

För att kunna testa och se webbapplikationen i realtid användes Angulars utvecklingsserver genom kommandot **ng serve**. Med hjälp av detta kunde projektet iterativt interageras med och se förändringar under utvecklingen.

Mycket av testningen underlättades av att TypeScript/Angular gav felmeddelanden vid kod som inte stämde överens med vad som förväntades eller inte fungerade logiskt. Det gjorde det betydligt enklare att upptäcka och rätta fel tidigt i utvecklingsprocessen.

Övrigt har webbplatsens responsivitet testats genom att använda olika webbläsare (Chrome, Edge samt FireFox).

4 Konstruktion

Själva uppbyggnaden av webbplatsen inleddes alltså med att skapa en utvecklingsmiljö med Angular. För att snabbt kunna komma i gång skapades därefter de två huvudkomponenterna för webbplatsen, därefter navigeringskomponenten. Därefter inleddes arbetet med resterande funktionalitet för webbplatsen.

4.1 Hantering av data (JSON-fil och service)

Kursinformationen utgår ifrån en lokal JSON-fil som ligger i projektets assets-mapp. Denna fil innehåller all kursdata från 2023 som används i applikationen.

För att hämta och hantera denna data skapades en Angular-service som använder HttpClient. Servicen ansvarar för att läsa in JSON-filen och tillhålla kursinformation till komponenterna i applikationen. Hämtningen sker med hjälp av Angulars HttpClient och returnerar kursinformation genom metoden `getCourses()`:

```
getCourses(): Observable<CourseModel[]>{  
  
    return this.http.get<CourseModel[]>(this.url);
```

Ytterligare en service skapades för att hantera localStorage. Den innehåller metoder för att lägga till och ta bort kurser som används av de två huvudkomponenterna.

För att vara säker på att kursinformationens data hanteras korrekt i applikationen användes ett TypeScript-interface för kursobjekten. Den definierar vilka egenskaper en kurs ska ha – som kurskod, namn, poäng med mera – och gör att risken för felaktig typning och inkonsekvent data minskar.

Kursdatan som hämtas används sedan i komponenterna för att skapa kurslistan i kursöversikten samt möjliggöra vidare hantering i ramschemat.

4.2 Navigering och routing

För att webbplatsen ska kunna navigera mellan de olika sidorna har Angular Routing använts. Routing är konfigurerad på så vis att användaren kan navigera mellan kursöversikten och ramschemat via definierade routes. Vid start av applikationen dirigeras användaren automatiskt till kursöversikten. Vid navigering byts den aktuella komponenten ut utan att hela sidan behöver laddas om:

Exempel på routing-konfiguration, placerat i `app.routes.ts`:

```
export const routes: Routes = [  
  
    { path: '', redirectTo: 'courses', pathMatch: 'full' },  
  
    { path: "courses", component: CoursesComponent }
```

Navigeringen mellan sidorna sker via en gemensam navigeringskomponent där Angulars routerLink används:

```
<a routerLink="/courses" routerLinkActive="active">Kurser</a>
```

Användaren kan råka skriva fel URL. Därmed har ytterligare en komponent skapats i projektet för att kunna visa en 404 felsida:

```
{ path: "**", component: PageNotFoundComponent, }
```

För att förenkla navigeringen mellan sidorna skapades en gemensam navigationskomponent. Den gör att samma meny kan användas på flera sidor utan att behöva duplicera samma kod i varje komponent.

4.3 Kursöversikt och Ramschema

Kursöversikten och ramschemat består av varsin tabell. Gemensamt för dem båda är att sortering sker vid klick på kolumn namnet för att välja sorteringsnyckel. Ytterligare klick växlar sortering mellan stigande och fallande ordning.

Navigeringen mellan dem båda fungerar enligt routerLink och path som visas ovan. Detta gör att bytet av sida sker mer smidigt då det är komponenterna som byts ut, inte hela webbplatsen.

4.3.1 Kursöversikt

Filtrering av kursdata i kursöversikten hanteras genom ett computed signal som returnerar en filtrerad lista baserat på valt ämne. En lokal signal används för att lagra valt ämne från ett select-element i HTML. När värdet ändras uppdateras den beräknade listan automatiskt utan att originaldatan påverkas.

I kursöversikten behövdes det möjlighet att välja kurser som ska visas i ramschemat. För att göra detta har en service skapats som hanterar tillägg och borttagning från localStorage.

Innan kurs kan läggas till genomförs en kontroll för att säkerställa att inga dubletter kan skapas:

```
const exists = courses.some( c => c.courseCode === course.courseCode );
```

Skulle det visa sig att den existerar, avbryts processen där. Annars fortsätter koden och skapar en ny uppdaterad array där den nya kursen läggs till:

```
const updated = [...courses, course];
```

Därefter sparas den uppdaterade listan i localStorage och används senare i ramschemat.

När användaren lägger till en kurs visas ett meddelande ovanför tabellen som meddelar användaren om kursen gick att lägga till eller om den redan finns i ramschemat.

Vid tillägg av en kurs uppdateras ett värde i komponenten som därefter visas i gränssnittet genom en paragraf i kursöversikten. På samma sätt beräknas antalet visade kurser vid filtrering dynamiskt.

För att hantera detta har Angulars computed använts, vilket gör att värden automatiskt uppdateras när datan förändras utan att manuell uppdatering krävs.

I TypeScript-filen:

```
scheduleCount = computed(() =>
    this.coursesService.savedCourses().length);
```

I HTML:

```
<p>{{ scheduleCount() }}</p>
```

Under projektets gång upptäcktes att sidan kunde bli långsam – det tog ett tag innan kurserna kunde visas. För att förhindra detta implementerades paginering så att endast ett begränsat antal kurser visas per sida. Användaren kan därefter bläddra mellan sidor med nästa/föregående knappar.

4.3.2 Ramschema

Ramschemat hämtar sin data från localStorage. När sidan laddas läses den sparade kurslistan in och lagras i en signal för att möjliggöra att automatisk uppdatering sker i UI vid eventuella förändringar.

Användaren kan också ta bort kurser från ramschemat. Då filtreras kursen bort och både signalen och localStorage uppdateras så att allt sparas direkt. Innan en kurs tas bort får man upp en bekräftelse, ifall man skulle ha råkat klicka fel.

Ramschemat har inte funktionalitet för filterning eller sökning, då fokus i denna komponent har legat på att hantera användarens redan valda kurser. Däremot finns det en funktionalitet som beräknar antal högskolepoäng av samtliga valda kurser.

5 Resultat

I slutet av projektarbetet har en fungerande webbapplikation skapats. Applikationen består av en kursöversikt och ett ramschema.

I kursöversikten finns det funktionalitet för att sortera, filtrera efter ämne samt söka efter specifika kurser. Dessa funktioner gör det lättare för användaren att hitta relevanta kurser från ett stort kursutbud.

Användaren kan även lägga till valda kurser i ett personligt ramschema. Detta gör det möjligt för användaren att stegvis bygga upp exempelvis en personlig studieplan utan att tappa överblicken över valda kurser.

Ramschemat visar samtliga valda kurser och gör det möjligt att bort kurser samt se det totala antalet högskolepoäng för de valda kurserna. Valda kurser sparas i webbläsaren med hjälp av localStorage vilket gör att användarens val sparas mellan sidoomladdningar eller om sidan stängs ner. Det gör applikationen mer praktiskt användbar över tid förutsatt att webbplatsens historik inte nollställs.

För att ge användaren tydlig återkoppling utan att behöva kontrollera i ramschemat så visas meddelanden när en kurs läggs till, eller om kursen redan finns i ramschemat. Dessa direkta meddelanden minskar risken för missförstånd och gör det tydligare för användaren vad som sker.

Två olika services har skapats för att hantera logik i webbapplikationen. Den ena ansvarar för att läsa in kursdata från en JSON-fil samt hantering av tillhörande TypeScript interface. Den andra hanterar ramschemat genom att tillhandahålla funktioner för att lägga till och ta bort kurser från localStorage.

Kursdata från en JSON-fil (2023 års kursutbud) visas i kursöversikten och visas uppdelad i sidor om 15 kurser per sida. Detta förbättrar överskådligheten och gör det möjligt för användaren att navigera bland många kurser utan att behöva ladda in hela datasetet på en gång.

Utöver detta är det möjligt för användaren att se hur många kurser som redan har lagts till i ramschemat, vilket ger en snabb överblick över studiplanen. Informationen uppdateras i realtid när kurser läggs till eller tas bort. Detta gör det enkelt att planera studier och se hur många poäng som totalt ingår i ramschemat.

Dessutom visas antalet kurser som hittats vid filterning eller sökning, vilket ger en tydligare återkoppling på användarens interaktion.

Webbplatsen har designats med responsiv design för att fungera på olika skärmstorlekar. Dessutom har testning genomförts i flera moderna webbläsare för att säkerställa att funktionalitet och utseende är konsekvent, oavsett vilken plattform användaren väljer att använda.

För att hantera den stora mängden kursdata har tabellerna i kursöversikten fått horisontell scrollning, vilket gör att alla kolumner fortfarande visas även på mindre skärmar. På mobila enheter är det endast tabellerna som blir scrollbara, medan övriga delar av sidan behåller sin layout. Detta säkerställer att informationen förblir tillgänglig utan att sidan tappar sin funktionalitet.

6 Analys

I introduktionen presenterades en problemformulering med de mål som fanns med denna uppgift. I detta kapitel utforskas huruvida dessa problem har blivit lösta, eventuella andra problem som upptäcktes under utvecklingen samt

6.1 TypeScript och Angular

Användandet av TypeScript under projektets gång har både varit hjälpsamt samt mer utmanande än med JavaScript, av den enkla anledningen att på grund av TypeScripts natur att vara mer strikt har fel kunnat upptäckas tidigt. Det har gjort det lite mer tidskrävande att identifiera exakt vad som har gått fel samt hur det ska gå att åtgärda.

Men den extra tydligheten har varit värdefull av exakt samma anledning. När TypeScript har påpekat att en kodrad har varit fel så visas exakt vilken del det handlar om: du som programmerare behöver inte gissa alltför mycket. Det har varit särskilt användbart när hanteringen av kursdata skulle implementeras.

Att använda Angular som ramverk upplevdes mer komplex än att skapa ett projekt utan alla medföljande filer som Angular automatiskt genererar. Det har tagit tid att bekanta sig med strukturen och förstå hur de olika delarna hänger ihop: hur komponenter, services och routing samarbetar.

Ibland upplevdes filstrukturen som uppdelad på ett sätt som gjorde det svårt att snabbt hitta relevant kod. Exempelvis uppstod en situation där en siländring hade påverkat layouten, men det var svårt att identifiera vilken komponent eller mapp som var berörd.

Efterhand som projektet har utvecklats har denna struktur blivit tydligare, och det är enklare att se fördelarna som Angular bidrar med: en färdig utvecklingsmiljö som går att utveckla vidare utan att behöva skapa alla mapper och filer som behövs.

6.2 Struktur av webbplatsen

Webbapplikationen har i slutet av projektet 2 huvudkomponenter som utgör de sidor som behövdes för uppgiften. De innehåller tabeller som visar kursöversikt respektive ramschema, varvid kursöversikten även har paginering implementerad för att underlätta inläsning av data.

Ytterligare en komponent har implementerats för att hantera navigationen mellan huvudkomponenterna samt för att inte behöva duplicera kod. Genom routing tillsammans med routerLink sker navigeringen mellan sidorna utan att hela webbplatsen laddas om – istället byts enbart den aktuella komponenten ut i vyn. Detta har bidragit till en mer responsiv applikation och skapar en smidigare användarupplevelse jämfört med traditionell sidladdning.

En 404-komponent har skapats för att hantera eventuella felskrivningar av URL.

Två service komponenter har skapats för att hantera viss logik på webbplatsen: en som hanterar inladdning av kursdata från en JSON-fil samt en service som hanterar användande av localStorage.

Mot slutet av projektet upptäcktes att enbart en service hade använts. Då all funktionalitet redan fanns direkt i huvudkomponenternas TypeScript-filer blev det en utmaning att avgöra vad som skulle passa att flyttas till en service och vad som borde finnas kvar i komponenterna för att hantera användargränssnittet.

Valet blev att skapa en service som ansvarar för att lägga till och ta bort kurser från localStorage.

I efterhand kan jag se hur viktiga och effektiviserande service kan vara i ett Angular/TypeScript projekt. Det är lättare att särskilja vad som är metoder för logik som går att återanvända i hela appen, samt vad som är uttryckligen UI-gränssnitt för komponenten.

6.3 Funktionalitet

Den funktionalitet som formulerades i problemformuleringen hade att göra med möjlighet till att söka efter kurser, filtrera baserat på ämne samt sortera i fallande/stigande ordning.

Denna funktionalitet finns på plats på kursöversiktens sida, men enbart sortering är implementerad på ramschemats sida. Detta var ett medvetet val som grundade sig i att ramschemat normalt innehåller betydligt färre kurser än den fullständiga kursöversikten. Sökning och filteringen ansågs därför inte ge samma nytta där som i kursöversikten.

Samtidigt fungerar sortering för att kunna sortera kurserna i bokstavsordning vilken bör göra det lättare för användaren att överskåda tabellen.

Ett par funktioner – antal kurser som finns i ramschemat, totalt antal högskolepoäng samt att kunna utläsa totalt antal listade kurser beroende på filterning/sökning – implementerades först mot slutet. Detta berodde på att alla krav för uppgiften inte hade identifierats under ett tidigt skede.

Dessa funktioner gick att implementera utan större problem med hjälp av computed, som räknar ut funktionerna dynamiskt. Men att det implementerades så sent pekar på vikten av att noggrant gå igenom samtliga krav innan och under utvecklingen av projektet. Det hade kunnat minska stress och effektivisera arbetet med projektet om dessa hade upptäckts i tid.

Även hanteringen av kursdata innebar vissa designval under utvecklingen. Det skapade interfacet innehåller samtliga fält från JSON-datan, trots att alla inte användes i den färdiga applikationen. Detta var ett medvetet val för att

behålla tillgång till all tillgänglig information och underlätta eventuella framtida utökningar. Samtidigt innebar det att delar av datamodellen aldrig användes i projektets nuvarande omfattning.

6.4 In i framtiden

En viktig lärdom är hur viktigt det är att läsa uppgiften ordentligt, kanske även skapa en egen punktlista med alla krav som uppgiften kräver. Hade detta gjorts från första början hade vissa delar som krävs för uppgiften inte upptäckts så sent som det faktiskt gjorde och arbete hade nog kunnat planeras mer effektivt.

Inför framtida projekt hade det varit bra att tidigt skapa en tydligare överblick när man jobbar i större ramverk som Angular. Eftersom många filer och mappar skapas automatiskt, och fler tillkommer under utvecklingen, kan projektet snabbt kännas stort och rörigt. Detta gäller särskilt när man inte har arbetat i projektet på ett tag, eller när många delar tillkommer samtidigt.

Detta visar på vikten av att regelbundet orientera sig i projektets struktur för att komma ihåg vad som gör vad och undvika att arbetsmiljön upplevs som rörig.

Personligen ser jag fram emot att utveckla mina kunskaper med TypeScript då det har varit roligt att använda en "annan" version av JavaScript: att den ger direkt återkoppling i utvecklingsmiljön och att framtida problem upptäcks mycket tidigare har varit roligt. All tack till JavaScript, men det är ganska skönt att få omedelbar vägledning.

7 Källförteckning

- [1] Ramirez L Jr. Beginner's Guide to Angular Signals [Internet]. Zero To Mastery; 2025 Apr 07 [citerad 2026-05-16]. Tillgänglig från: <https://zerotomastery.io/blog/angular-signals/>
- [2] Zayed M. Exploring the New Control Flow Syntax in Angular 17 [Internet]. DEV Community; 2023 Nov 19 [citerad 2026-05-16]. Tillgänglig från: <https://dev.to/mariazayed/exploring-the-new-control-flow-syntax-in-angular-17-339g>
- [3] TypeScript Team. TypeScript for the New Programmer [Internet]. 2026 [citerad 2026-06-04]. Tillgänglig från: <https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html>
- [4] GeeksForGeeks. What is Angular? [Internet]. 2025 Jul 23 [citerad 2026-06-06]. Tillgänglig från: <https://www.geeksforgeeks.org/angular-js/what-is-angular/>